Comparison between CPU and GPU performance in barcode detection

Tiago Santos
Departamento de Engenharia Eletrotécnica (DEE)
Instituto Superior de Engenharia do Porto (ISEP)
Porto,Portugal
1181367@isep.ipp.pt

Rui Mendes
Departamento de Engenharia Eletrotécnica (DEE)
Instituto Superior de Engenharia do Porto (ISEP)
Porto,Portugal
1181359@isep.ipp.pt

***Abstract*—Object detection is a crucial task in various applications, including retail, logistics, and industrial automation. With the growing demand for efficient real-time object detection, the usage of GPU cards paired with cutting-edge computing provides AI capabilities for embedded vision tasks. This work explores the comparison between CPU and GPU performance in barcode detection, evaluating traditional and deep learning algorithms through inference times and object detection metrics. After identifying the most efficient algorithm, a simulated warehouse scenario was developed in Webots to validate its applicability in a near-real environment. Finally, a real-time validation with a webcam was performed, comparing CPU and GPU execution in terms of frames per second (FPS) and detection fluency.**

***Index Terms*—Barcode detection, CPU, GPU, Inference, Real-Time**

I. Introduction

A. Background

Barcode detection plays a crucial role in numerous areas related to industrial automation. Since inventory management to local recognition, the ability to accurately and efficiently detect barcodes in real-time is crucial for these operations. Traditional object detection methods often rely on classic image processing, which requires perfect conditions, and in real-time scenarios those cannot be guaranteed [1], [2].

The beginning of barcode detection and other computer vision tasks, solely relied on CPUs running traditional image processing algorithms, which were efficient under ideal conditions but struggled in real-time scenarios [3]. Prior to the early 2000's, the CPU served as the primary computational unit, while the GPU was dedicated solely to rendering graphics. This paradigm shift with the introduction of general-purpose programming frameworks, specifically NVIDIA's CUDA in 2006, which enabled GPUs to be leveraged for parallel computing workloads [4]. This development was the cornerstone of a new era in computer vision research by increasing the process's speed, such as convolutions and feature extraction. A critical turning occurred with the introduction of deep learning, in 2010s, particularly Convolution Neural Networks (CNNs), demonstrated superior robustness and accuracy over traditional methods [5]. Consequently, GPUs have become the standard hardware for both deep learning training and inference [6]. With the advancement of artificial intelligence (AI) and the increasing demand for real-time processing, it has led to the usage of Graphics Processing Units (GPUs) instead of Central Processing Units (CPU). Compared to the CPUs, GPUs excel in parallel processing. GPUs excel in parallel processing due to their amount of cores,making them optimal for handling the demands required by the newest algorithms. On the other hand, the CPUs have higher process frequency which is ideal for sequential processing [4]. The usage of AI algorithms requires edge computing as a result of real-time applications. Thus, embedded platforms such as the Qualcomm Robotics RB5, have emerged as powerful solutions, enabling object detection to be performed directly on devices, similarly to drones and quad-robots, reducing latency. The Qualcomm RB5 platform is equipped with an octa-core Qualcomm®Kryo ™585 CPU and a Qualcomm®Adreno™650 GPU, allowing a heterogeneous computing environment improved for AI and computer vision tasks [7].

B. Objectives

The main objective of this paper is to compare the performances between CPU and GPU in barcode detection regarding the methods used. That being said, it aims to:

- Study of the benefits of using GPU for barcode detection instead of CPU.
- Analyse the performance of Deep Learning algorithms and the traditional methods for object detection.
- Experimental comparison of inference times and accuracy metrics on CPU and GPU
- Optimization and implementation of a real-time barcode detection pipeline in the Webots with and without ROS2.
- Perform an additional real-time validation with a webcam, evaluating CPU and GPU execution in terms of average frames per second (FPS) and detection fluency.

C. Structure of the Report

This paper is organized as follows:

- **Section 2** states the problem in detail.
- **Section 3** describes both traditional and deep learning techniques studied to outcome the problem.
- **Section 4** reviews related work on the usage GPU acceleration in embedded systems instead of CPU.
- **Section 5** provides the types of GPU that can be used in object detection, according to the user's goal.

- **Section 6** Solution and results
- **Section 7** Conclusion and discussion of results

II. Problem and Proposed Solution

A. Problem

Barcode detection is fundamental in logistics, warehouse automation, and retail, where real-time performance and robustness under variable conditions are critical. Traditional computer vision techniques perform adequately under ideal scenarios but degrade significantly in the presence of noise, distortions, or suboptimal illumination. Deep learning–based detectors mitigate these limitations, though at the expense of substantial computational requirements.

The selection of hardware is therefore pivotal: CPUs are well-suited for sequential processing, whereas GPUs provide extensive parallelism that substantially accelerates inference but with increased energy consumption, limiting their suitability for embedded platforms. The central challenge lies in determining the optimal trade-off between inference speed and detection accuracy in barcode recognition tasks.

B. Proposed Solution

To address this problem, a benchmark study was performed using both traditional and deep learning methods for barcode detection. A publicly available annotated dataset from Roboflow, containing training, validation, and test splits, was used to ensure reproducibility. The following detection algorithms were trained and evaluated:

- **YOLOv8:** real-time single stage detector;
- **Faster R-CNN:** two stage detector with high accuracy;
- **Single Shot Multibox Detector (SSD):** single stage detector balancing speed and accuracy;
- **RetinaNet:** single stage detector with focal loss for class imbalance.

Each algorithm was tested on CPU and GPU configurations, and results were compared in terms of inference time, Intersection over Union (IoU), and Average Precision at 0.75 (AP@0.75). Another comparison was made using Pyzbar library (OpenCV) but since it only was possible to test using CPU this option was discarded which rely on traditional methods and its results were quite poor.

Based on this comparison, the best performing algorithm was selected and validated in a Webots simulation replicating a warehouse environment. Two simulation scenarios (worlds) were designed, both evaluated with and without ROS2 integration, to approximate a realistic deployment environment. This approach allowed not only a theoretical comparison between CPU and GPU performance, but also a practical validation of the selected model under simulated operational conditions.

III. CPU versus GPU

Traditional embedded systems have depended on CPUs for image processing tasks, together with the traditional object detection methods. Nevertheless, with the increased usage of AI for real-time tasks, GPUs were adopted. CPUs, where the tasks are executed sequentially relying on the frequency, struggle on the parallel processing requirements of computer vision and Deep Learning. Contrarily, GPUs provide a massive number of cores, leveraging parallelism to speed up the workloads, making them fitting for vision-based and real-time applications, such as barcode detection [4].

A considerable amount of studies have investigated the performance trade-off between CPU and GPU for deep learning inference. An article [8] studied the difference of CPU and GPU performance for deep learning-based web application and discovered that GPUs give a 4 to 5 times speed-up over CPUs in all tests due to their parallel architecture. Furthermore, a paper [6] benchmarked object detection efficiency on embedded hardware platforms and determined that GPU significantly reduces inference time while assuring power efficiency, making it a reliable option for real-time applications.

Regardless of these advantages, GPUs have a higher power consumption compared to CPUs, which can be a restraining factor for embedded systems. A paper [9] made a comparison of YOLOv5 object detection models, evaluating inference time, power and energy consumption across CPUs, GPUs and NPUs. The study demonstrated that GPUs, in spite of having the fastest inference times, NPUs showed better energy efficiency, making a decent alternative for low power applications.

Nevertheless, for real-time tasks where the performance must be high, GPUs remain the best choice.

IV. Traditional Techniques vs Deep Learning Techniques

A. Traditional Techniques

The usage of traditional techniques relies mainly on image processing techniques such as edge detection, morphological operations, threshold, and feature matching. The early methods used to detect barcodes studied for this paper were:

- **Methods based on Mathematical Morphology** - with the usage of dilation and erosion, linear patterns are detected on barcodes. Method studied by Katona and Nyul to detect barcodes in images [3].
- **Hough Transform** - on a study made by Zamberletti and others, the implementation of this techniques allowed the verification of barcode orientation, trough the detection of the bar borders [10].
- **Detection based of Gradients and Derivatives** - this technique has a moderated efficiency, since it presents problems with distortion and orientation of the barcodes. In addition, it can fail under poor lighting and noise. This study was promoted by Gallo and Manduchi [11].
- **Maximally stable extremal regions** - technique studied by Creusot and Munawar, it allows the identification of stable high contrast regions, mainly used on challenging environments [12].
- **QR codes detection (Viola-Jones Algorithm)** - Belussi and Hirata made use of the viola-jones algorithm and adapt it to locate patterns on images easier, increasing the efficiency during the detection [13].
- **Detection algorithm base of tiles** - in this study, Szentandrasi used Detection algorithm base of tiles togheter

with Histogram of Oriented Gradients(HOG), to do the segmentation and classification of different regions in a image [14].
- **Inverse Decoding** - this technique consists on locating Regions Of Interest (ROI). This paper made by Wachenfeld, demonstrated that this technique maintains this cycle process until it becomes validated [15].

These methods are effective to locate barcodes in perfect conditions, with the advantage of being low computational cost. Furthermore, they offer interpretable and deterministic behaviour caused by their simple mathematical operations. However, traditional techniques struggle when faced with awful conditions, such as bad lighting, motion blur, and perspective distortions, often displayed in real-time scenarios [5], [16]. Moreover, rule-based algorithms lack the skill to generalize independently of the barcode formats and orientations, requiring extensive manual tuning for each application [17].

B. Deep Learning Techniques

With the recent advancements in deep learning and convolutional neural networks (CNNs), object detection improved a lot, surpassing the limitations of traditional methods. Modern methods use end-to-end neural network architectures that learn object detection from large datasets, avoiding the need for extensive parameter tuning [5]. Unlike traditional algorithms, deep learning methods learn hierarchical features from data, which means, even if the barcode presents different orientations and formats, it can detect it even under harsh conditions such as the ones mentioned before [5], [18].

Despite these advantages, deep learning techniques require higher computational power. The training of deep neural networks requires GPU acceleration, making them inadequate for low power embedded systems [17]. However, sided with transfer learning and model quantization techniques, it reduces the computational overhead and enables their usage on embedded devices [18].

For this paper, the following methods were studied and analyzed to find out the appropriate one for the present problem:

- **YOLO** - this method begins by simultaneously predict bounding boxes and class labels, which allows reaching real-time performance [19]. Throughout his versions, in the recent one, YOLOv8, paired with EfficientViT and grouped convolutions, allowed the increasing of accuracy, while reducing computational costs, making it ideal for embedded systems [18], [20]
- **Single Shot Multibox Detector (SDD)** - algorithm that removes the region proposal steps like the traditional techniques and in a singles forward pass, predicts bounding boxes leading to a balance between accuracy and speed [17], [21].
- **FAST R-CNN** - as a two stage detector, firstly generates region proposals and then classifies objects. It reaches high accuracy but is computationally expensive. Even though it is suitable for barcode detection on controlled environments , its inference time makes it unusable for real-time applications [22], [23].
- **RetinaNet** - with the introduction of a new focal loss function, RetinaNet improves the training step of one stage methods on imbalanced dense data. This method addresses the class imbalance problem in object detection [17].
- **Mask R-CNN** - this method adds to Faster R-CNN an additional branch. Mask R-CNN enables the prediction of segmentation masks for each ROI, allowing for simultaneous object detection and instance segmentation. Its inference performance is high, making it well-suited for real-time applications [24].

V. Types of GPU used in object detection

As mentioned before, speed is a crucial factor for image processing and feature detection. The papers [25] and [26] verified that the GPU speed was eight times higher than the CPU, allowing drones to achieve real-time detection. Recently, the GPUs have a higher capacity for model development and training compared to CPUs:

- **NVIDIA RTX 30/40 Series** - ideal for development and training. Excellent for neural networks of average and high scale [27];
- **NVIDIA A100 ou H100** - high cost implementation, only appliable for huge scale training [28];
- **NVIDIA RTX 4090 ou Titan RTX** - used on high performance systems for processing large datasets or complex models [29];
- **Cloud GPUs** - cloud usage when there's no hardware powerful enough [30].

However, there are GPUs designed with the only purpose of being utilized in drones:

- **NVIDIA Jetson Series** - designed for cutting-edge AI, optimized for low power consumption and portability.
  - **Jetson Nano** - affordable, optimized for low-power tasks and portability [31];
  - **Jetson Xavier** - the most powerful, ideal for cutting-edge AI applications [32];
  - **Jetson Orin** - the latest generation for advanced detection and processing [33];
- **Coral Edge TPU** - ideal for drones with limited resources.
- **AMD Radeon GPUs** - an alternative to NVIDIA, supporting deep learning frameworks through ROCm [34];
- **Intel Movidius Neural Compute Stick** - USB-sized, lightweight, and capable of running models with OpenVINO [35].
- **Qualcomm RB5** - platform designed for robotics and drones, with a heterogeneous computing architecture, which allows efficient AI inference and computer vision at lower consumption [7].

VI. Solution and Results

A. Implementation

The implementation was divided into two main phases: algorithm benchmarking and simulation validation. Although batch processing can improve throughput on GPUs, in this work inference was performed image by image to better reflect real-time performance.

1) **Dataset and Preprocessing**
   - The dataset was obtained from Roboflow, already annotated with training, validation, and test splits.
   - Images were normalized and augmentations were applied such as blur, crop, rotation, exposure, shear and brightness to enrich the dataset for the training part.
2) **Training and Evaluation**
   - The deep learning models (YOLOv8, Faster R-CNN, SSD, RetinaNet) were trained.
   - Training was performed for 20 epochs with a batch size of 4, using SGD optimizer and an initial learning rate of $1e^{-3}$ and 500 images were used.
   - The traditional method (Pyzbar) was directly evaluated without training, since it relies on image processing techniques.
   - Each model was tested on both CPU and GPU to measure inference times and accuracy.
3) **Metrics**
   - Inference time (average of all images, in ms)
   - IoU (Intersection over Union) between predicted and ground-truth bounding boxes. Performed as average of all images;
   - AP@0.75 (Average Precision at IoU threshold 0.75) as an accuracy indicator. Performed as average of all images

B. Hardware configuration

To ensure that the results were not biased by a single hardware platform, tests were conducted on three different PCs with distinct specifications:

TABLE I: Hardware specifications of tested PCs

Device	CPU	GPU	VRAM
PC 1	Intel Core i7-13650HX	NVIDIA RTX 4060	8GB
PC 2	Intel Core i7-13700K	NVIDIA RTX 3070	8GB
PC 3	Intel Core i5-8300H	NVIDIA GTX 1050	4GB

C. Results

After the training and evaluation process, inference results and accuracy metrics were obtained for each of the models tested on different hardware configurations. The values are an average of the 100 images.

Table II shows the results obtained on PC 1 (Intel i7-13650HX + RTX 4060 8GB). It can be observed that YOLOv8 achieved the best compromise between inference time and accuracy, with only 30.32 ms on the GPU and an IoU of 89.92%, in addition to an AP of 0.9789. On the other hand, Faster R-CNN, although it achieved the highest AP (0.9993), had significantly higher times (1077.42 ms on the CPU and 64.98 ms on the GPU), making it unfeasible for real-time applications.

TABLE II: Inference and accuracy comparison - **PC 1**

Algorithm	CPU Time (ms)	GPU Time (ms)	IoU (%)	AP
Yolov8	79.13	30.32	89.92	0.9789
Faster R-CNN	1077.42	64.98	76.35	0.9993
SSD	177.45	64.98	83.91	0.9967
RetinaNet	903.47	58.50	71.93	0.9686

Table III shows the results for PC 2 (Intel i7-13700K + RTX 3070 8GB). Consistently, YOLOv8 was the fastest model (20.80 ms on the GPU), confirming its real-time detector character. The SSD also demonstrated balanced performance (32.39 ms on the GPU with good accuracy), but inferior to YOLOv8.

TABLE III: Inference and accuracy comparison - **PC 2**

Algorithm	CPU Time (ms)	GPU Time (ms)	IoU (%)	AP
Yolov8	57.10	20.80	89.92	0.9789
Faster R-CNN	1031.17	56.13	76.35	0.9993
SSD	119.32	32.39	83.91	0.9967
RetinaNet	860.48	67.28	71.93	0.9686

In the case of PC 3 (Intel i5-8300H + GTX 1050 4GB), shown in Table IV, there is a significant difference: inference times increased in all algorithms due to the limitations of the older GPU. Even so, YOLOv8 remained in the lead with 99.83 ms, although above the threshold considered 'real time' (<33 ms). This demonstrates that the choice of GPU directly impacts the viability of the application in practical scenarios.

TABLE IV: Inference and accuracy comparison - **PC 3**

Algorithm	CPU Time (ms)	GPU Time (ms)	IoU (%)	AP
Yolov8	148.73	99.83	89.92	0.9789
Faster R-CNN	3173.13	890.27	76.35	0.9993
SSD	529.38	301.53	83.91	0.9967
RetinaNet	3238.92	1002.12	71.93	0.9686

Table V summarises the energy consumption during the tests. Although GPUs consume more instantaneous power, the reduced execution time made them more energy efficient overall for batches of 100 images. A clear example is YOLOv8, which consumed only 0.000065 kWh on the GPU, compared to 0.000130 kWh on the CPU.

TABLE V: Energy consumption while performing inference test

Algorithm	CPU (W / kWh)	GPU (W / kWh)
Yolov8	05.57 / 0.000130	13.76 / 0.000065
Faster R-CNN	06.95 / 0.001154	45.19 / 0.000340
SSD	03.32 / 0.000338	12.48 / 0.000188
RetinaNet	04.43 / 0.000949	42.33 / 0.000315

Finally, Figure 1 illustrates an example of barcode detection using the YOLOv8 model. The image shows bounding boxes accurately placed around the detected barcodes. This visual confirmation validates the numerical results obtained in the benchmarking phase, demonstrating that YOLOv8 is capable of robustly detecting barcodes under realistic image conditions. The bounding boxes remained stable across variations in orientation and moderate illumination changes, reinforcing YOLOv8's suitability for real-time warehouse scenarios.

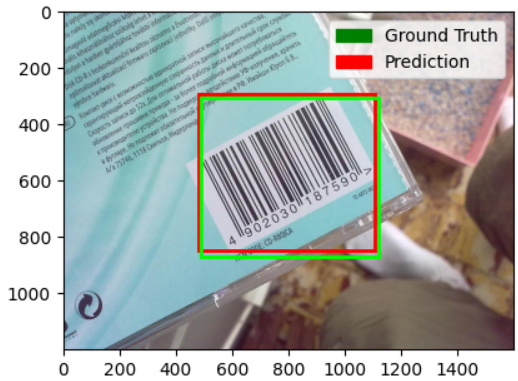

Fig. 1: Example of a YOLOv8 detection result

D. Simulation

To verify the performance of the selected YOLOv8 algorithm under real-time conditions, a simulation was constructed using the Webots platform. Webots, an open-source robot simulator, was selected to emulate a warehouse scenario for this study. Its physics-based 3D environment provides realistic sensor simulation and integrated libraries for robotic components. Furthermore, its native Python support enables seamless integration with deep learning frameworks, allowing for efficient testing of the YOLOv8 pipeline [36]. Unlike Gazebo, which often requires complex, ROS-dependent configurations, Webots facilitates faster prototyping and direct algorithm validation, making it the more suitable choice for this case study.

1) Webots: To validate the performance of the selected YOLOv8 model in a near-realistic environment, two simulation scenarios were developed in Webots: one using only the native Webots environment, and another integrated with ROS2 for greater flexibility and interoperability with robotic systems. In both scenarios, a Crazyflie drone was controlled by an autonomous flight algorithm combining a PID controller for stabilization and navigation, and a YOLOv8-based barcode detector running in real-time. The PID controller is responsible for maintaining the drone's altitude and latitude, while YOLOv8 processes images from the onboard camera to detect and localize barcodes in the environment.

The following Algorithm 1 describes the core logic of the controller implemented in Webots, integrating real-time detection and flight control.

Algorithm 1 Real-Time Drone Control with Barcode Detection

1: Initialize robot, sensors, and YOLO model
2: Configure PID controller and flight parameters
3: **while** simulation is running **do**
4: Read sensor data (IMU, GPS, camera)
5: Process keyboard input for manual control
6: Capture image from camera
7: Convert image to OpenCV format
8: $results \leftarrow$ model($frame, device =$ 'gpu') ▷ Run YOLOv8 inference
9: **for all** $box \in results.boxes$ **do**
10: Draw bounding box and label on $frame$
11: **end for**
12: Display $frame$
13: Calculate motor commands via PID controller
14: Apply velocities to motors
15: Update previous state variables
16: **end while**

The algorithm illustrates a tight integration of perception and control. The loop begins by reading data from the inertial measurement unit (IMU), GPS, and gyroscope (Line 4). User commands from the keyboard are captured to guide the drone's movement (Line 5). The key step for barcode detection occurs on Line 8, where each captured camera frame is processed by the pre-trained YOLOv8 model. The resulting detections are then used to annotate the video stream for monitoring purposes.

Subsequently, the PID controller calculates the necessary motor velocities based on the current state of the drone and the user's input (Line 13). These commands are applied to the motors (Line 14), completing one iteration of the control loop. This design ensures low-latency response, which is critical for real-time operation. It is important to note that for the ROS2-based simulation, this monolithic loop was decoupled into separate nodes publishing and subscribing to topics such as `/camera/image_raw` and `/cmd_vel`, improving modularity and interoperability with other robotic systems.

2) Webots with ROS2: To enhance modularity and interoperability with real robotic systems, the simulation was implemented using ROS2 (Robot Operating System 2). The architecture was decoupled into separate nodes, each with a distinct responsibility, communicating through topics. This design comply to core ROS2 principles, allowing for independent development, testing, and replacement of components.

The system comprises three key elements: a PID controller library for flight stabilization, a driver node interfacing with Webots, and a teleoperation node for manual control and real-time detection.

The driver node publishes camera images on the `/crazyflie/camera/image_raw` topic. The teleoperation node subscribes to this topic to perform detection and publishes velocity commands on the `/cmd_vel` topic, to which the driver node subscribes to control the drone (Figure 2). The PID controller is imported as a library by the driver node to calculate motor commands.

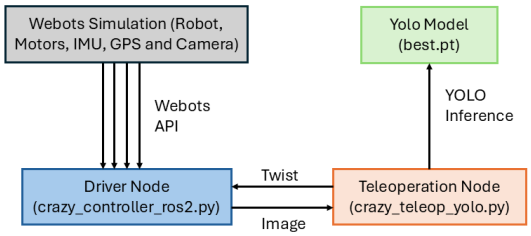

Fig. 2: Data flow diagram of the ROS2-based simulation architecture.

This architecture demonstrates a classic robotics pattern of separating perception, decision-making, and actuation into distinct processes.

The **Driver Node** acts as the low-level hardware abstraction layer, responsible for all direct interaction with the Webots simulation. Its primary role is to execute the control loop: reading the current state of the drone, using the PID controller (Algorithm 3) to compute the necessary motor commands, and applying them. This PID controller implements a cascaded structure, translating high-level velocity commands into stable low-level motor thrusts through nested attitude and rate loops.

The **Teleoperation Node** represents the high-level decision-making and perception layer. It handles user input via the key bindings defined in Table VI and performs real-time barcode detection using YOLOv8 on images received from the driver node. It sends goal commands (`/cmd_vel`) without needing knowledge of the low-level stabilization.

Key	Action (Twist Message)
`UP Arrow`	Forward (+linear.x)
`DOWN Arrow`	Backward (-linear.x)
`LEFT Arrow`	Strafe Left (+linear.y)
`RIGHT Arrow`	Strafe Right (-linear.y)
`W`	Ascend (+linear.z)
`S`	Descend (-linear.z)
`Q`	Rotate CCW (+angular.z)
`E`	Rotate CW (-angular.z)
`X`	Emergency Stop (Zero all)

TABLE VI: Keyboard bindings for teleoperation.

The main advantage of this design is its modularity. The perception algorithm (YOLO) can be improved or replaced without affecting the control code. Similarly, the PID gains can be tuned or a different control strategy tested without modifying the perception or user interface code. This separation of

Algorithm 2 Integrated ROS2 Control and Perception System

Driver Node: `crazy_controller_ros2.py`
Initialize ROS2 node `crazyflie_driver`
Initialize Webots `Robot()`, motors, IMU, GPS, camera
Import `PID_Controller` from `pid_controller.py`
Create publisher: `Image` → `/crazyflie/camera/image_raw`
Create subscriber: `/cmd_vel` → `cmd_vel_callback`

Teleoperation Node: `crazyflie_yolo_teleop.py`
Initialize ROS2 node `crazyflie_yolo_teleop`
model ← YOLO('best.pt') ▷ Load trained model
Create publisher: `Twist` → `/cmd_vel`
Create subscriber: `/crazyflie/camera/image_raw` → `image_callback`

Main Loop (Parallel Execution)
while rclpy.ok() **do**
▷ - Teleop Node Thread -
  key ← getKey() ▷ Non-blocking keyboard input
  twist_msg ← MapKeyToTwist(key)
  Publish twist_msg to /cmd_vel

▷ — Driver Node Thread —
  robot.step(timestep) ▷ Step the Webots simulation
  Read sensors: $roll, pitch, yaw, altitude, vx, vy$
  Check for command timeout
  motors ← PID.pid(dt, desired_vx, ..., actual_roll, ...)
  SetMotorVelocities(motors)
  frame ← Camera.GetImage()
  Publish frame to /crazyflie/camera/image_raw

▷ — Teleop Node Callback (Async) —
  procedure IMAGECALLBACK(msg)
    frame ← CVBridge(msg) ▷ Convert ROS msg to OpenCV
    results ← model(frame) ▷ Run YOLOv8 inference
    annotated_frame ← results[0].plot()
    cv2.imshow(annotated_frame)
  end procedure

  rclpy.spin_once() ▷ Process all callbacks
end while

Algorithm 3 High-Level PID Control Function

```
function PID(desired_vx, desired_vy, desired_altitude, sensor_data)
    Input: Desired velocities, altitude, and current sensor data
    Output: Motor commands for m1, m2, m3, m4

    Calculate errors between desired and actual state
    Compute control signals for attitude and altitude
    Mix control signals into individual motor commands
    Limit motor commands to valid range [0, 600]

    return [m1, m2, m3, m4]
end function
```

concerns, facilitated by ROS2, is a best practice in robotics software engineering and was successfully validated in the Webots simulation environment, providing a robust platform for barcode detection.

E. Extra Validation: Real-Time Inference

To further validate the trained YOLOv8 model beyond the dataset and simulation results, real-time inference tests were conducted using a standard webcam and everyday products containing barcodes. The model was executed on both CPU and GPU, and the average frames per second (FPS) was measured across the entire session.

When executed on the CPU, the system achieved few FPS (Figure 3). Although detections were still possible, the low frame rate resulted in less fluent updates, which could limit practical deployment in robotic platforms requiring continuous perception.

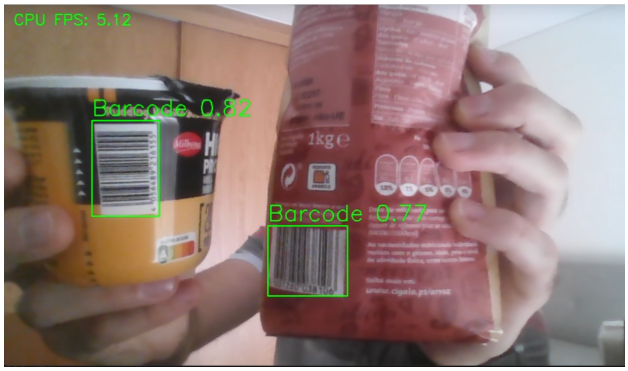

Fig. 3: Example of real-time barcode detection with YOLOv8 running on the **CPU**

In contrast, when executed on the GPU, the system reached higher FPS values (Figure 4), enabling smooth real-time detection with bounding boxes consistently drawn around the barcodes.

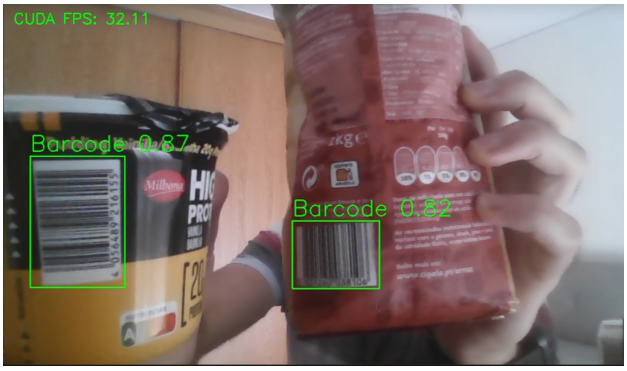

Fig. 4: Example of real-time barcode detection with YOLOv8 running on the **GPU**

This performance improvement highlights the importance of GPU acceleration for applications such as drones or mobile robots operating in warehouse environments, where low-latency perception is essential.

VII. Conclusion

A. Discussion of Results

The results obtained allow for a clear comparison of CPU and GPU performance across the different models tested. Overall, YOLOv8 stood out as the most balanced algorithm, achieving significantly lower inference times than the others without compromising accuracy. In all scenarios, it maintained an IoU of 89.92% and an AP of 0.9789, ensuring robustness and consistency.

Faster R-CNN had the highest accuracy (AP of 0.9993), but extremely high inference times, especially on the CPU, compromise its use in real-time applications. This shows that, although it is a robust model, its complexity makes it unfeasible in environments that require fast response times.

SSD proved to be an intermediate alternative, with moderate inference times and accuracy close to YOLOv8, making it a viable option in systems where a compromise between speed and accuracy is sought. RetinaNet, on the other hand, revealed the worst overall performance, with high times and lower accuracy values, making it uncompetitive compared to the other algorithms.

To validate the practical applicability of these findings, the selected YOLOv8 model was integrated into a simulated warehouse environment using Webots. Two distinct scenarios were developed: a standalone Webots simulation with a monolithic control and perception architecture, and a more advanced, modular implementation using ROS2. In both scenarios, a Crazyflie drone was tasked with navigating the warehouse and detecting barcodes in real-time. The simulation confirmed that the inference speeds and accuracy observed in the benchmark tests were sufficient for real-time operation under dynamic conditions. Furthermore, the transition to a ROS2-based architecture demonstrated significant improvements in system modularity, interoperability, and ease of integration with higher-level autonomy stacks, closely mirroring real-world robotic systems. This step provided strong evidence

that the YOLOv8 model, coupled with appropriate hardware, is not only viable in theory but also effective in a near-real operational context.

Another relevant point is the comparison between CPU and GPU. The results show that the use of GPUs reduced inference times by three to ten times, depending on the algorithm and graphics card generation. However, it was found that on older GPUs, such as the GTX 1050, the performance gain was not sufficient to reach levels considered real-time, highlighting the importance of up-to-date hardware for this type of application.

Regarding energy consumption, although GPUs have higher instantaneous power, the reduced execution time resulted in lower total energy consumption in inference batches. YOLOv8 exemplifies this scenario well: despite higher consumption in Watts, the shorter execution time meant that the final energy expenditure was lower than that of the CPU.

In addition to the offline benchmarks and simulation tests, real-time webcam validation confirmed that GPU acceleration ensures fluid detections suitable for practical deployment. While CPU-only execution was insufficient for real-time, GPU inference consistently achieved smooth performance, which is essential in mobile robotic applications.

In summary, the results confirm that the choice of algorithm and hardware is crucial for the viability of real-time barcode detection. YOLOv8, combined with modern GPUs, proved to be the most suitable solution, combining speed, accuracy, and energy efficiency.

B. Future Work

As future work, the main objective will be to deploy the proposed detection pipeline on the Qualcomm RB5 platform, allowing the evaluation of inference performance and energy efficiency in a real embedded system. Another relevant step is to explore alternative hardware accelerators, such as NPUs or TPUs, in order to investigate more energy-efficient solutions for real-time deployment.

An additional step would be to replace manual teleoperation with autonomous navigation, allowing the drone to actively search for barcodes corresponding to items in a database, which represents a significant step toward full automation in inventory management systems.

Finally, moving from simulation to physical robotic platforms would enable a complete validation of the system in real warehouse environments, complemented by model optimization techniques such as quantization or pruning to further reduce computational costs.

REFERENCES

[1] PackageX, "Barcodes and logistics: Going beyond traditional scanning methods," 2024, accessed: 2025-02-08. [Online]. Available: https://packagex.io/blog/barcodes-and-logistics

[2] Qodenext, "The various applications of barcodes in the retail industry," 2024, accessed: 2025-02-08. [Online]. Available: https://qodenext.com/blog/the-various-applications-of-barcodes-in-the-retail-industry/

[3] M. Katona and L. G. Nyul, "Efficient 1d and 2d barcode detection using mathematical morphology," in *International Symposium on Mathematical Morphology and Its Applications to Signal and Image Processing*, 2013, pp. 464–475.

[4] E. Buber and B. Diri, "Performance analysis and cpu vs gpu comparison for deep learning," in *2018 International Conference on Computer Science and Engineering (CEIT)*. IEEE, 2018, pp. 1–6.

[5] A. B. Amjoud and M. Amrouch, "Object detection using deep learning, cnns and vision transformers: A review," *IEEE Access*, vol. 11, pp. 35 479–35 486, 2023.

[6] I. Lazarevich, M. Grimaldi, R. Kumar, S. Mitra, S. Khan, and S. Sah, "Yolobench: Benchmarking efficient object detectors on embedded systems," *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 1169–1177, 2023. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2023W/RCV/papers/Lazarevich_YOLOBench_Benchmarking_Efficient_Object_Detectors_on_Embedded_Systems_ICCVW_2023_paper.pdf

[7] Qualcomm, "Qualcomm robotics rb5 development kit," 2024, accessed: 2025-02-08. [Online]. Available: https://www.thundercomm.com/product/qualcomm-robotics-rb5-development-kit/

[8] E. Buber and B. Diri, "Performance analysis and cpu vs gpu comparison for deep learning," *2018 International Conference on Computer Science and Engineering (UBMK)*, pp. 306–311, 2018. [Online]. Available: https://www.researchgate.net/publication/334168063

[9] M. D. Abo, "An efficiency comparison of npu, cpu, and gpu when executing an object detection model yolov5," Master's thesis, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, June 2024, degree Project in Computer Science and Engineering.

[10] A. Zamberletti, I. Gallo, and S. Albertini, "Robust angle invariant 1d barcode detection," in *Pattern Recognition (ACPR), 2013 2nd IAPR Asian Conference on*, 2013, pp. 160–164.

[11] O. Gallo and R. Manduchi, "Reading 1d barcodes with mobile phones using deformable templates," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 9, pp. 1834–1843, 2011.

[12] C. Creusot and A. Munawar, "Real-time barcode detection in the wild," in *Applications of Computer Vision (WACV), 2015 IEEE Winter Conference on*. IEEE, 2015, pp. 239–245.

[13] L. F. F. Belussi and N. S. T. Hirata, "Fast qr code detection in arbitrarily acquired images," in *Graphics, Patterns and Images (Sibgrapi), 2011 24th SIBGRAPI Conference on*, no. September. IEEE, 2016, pp. 281–288.

[14] I. Szentandrasi, A. Herout, and M. Dubska, "Fast detection and recognition of qr codes in high-resolution images," in *Proceedings of the 28th Spring Conference on Computer Graphics*, 2013, pp. 129–136.

[15] S. Wachenfeld, S. Terlunen, and X. Jiang, "Robust recognition of 1-d barcodes using camera phones," in *Pattern Recognition, 2008. ICPR 2008. 19th International Conference on*, 2008, pp. 1–4.

[16] N. O'Mahony, S. Campbell, A. Carvalho, S. Harapanahalli, G. Velasco-Hernandez, L. Krpalkova, D. Riordan, and J. Walsh, "Deep learning vs. traditional computer vision," in *Advances in Computer Vision: Proceedings of the 2019 Computer Vision Conference (CVC)*. Springer Nature Switzerland AG, 2019, pp. 128–144. [Online]. Available: https://arxiv.org/abs/1910.13796

[17] P. Mittal, R. Singh, and A. Sharma, "Deep learning-based object detection in low-altitude uav datasets: A survey," *Image and Vision Computing*, vol. 104, p. 104046, 2020. [Online]. Available: https://doi.org/10.1016/j.imavis.2020.104046

[18] S. Srivastava, A. V. Divekar, C. Anilkumar, I. Naik, V. Kulkarni, and P. V., "Comparative analysis of deep learning image detection algorithms," *Journal of Big Data*, vol. 8, pp. 1–20, 2021. [Online]. Available: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-021-00434-w

[19] D. K. Hansen, K. Nasrollahi, C. B. Rasmussen, and T. B. Moeslund, "Real-time barcode detection and classification using deep learning," in *Proceedings of the 9th International Joint Conference on Computational*

Intelligence - Volume 1: IJCCI. SCITEPRESS Digital Library, 2017, pp. 321–327.

[20] F. N. Chen and Others, "A lightweight barcode detection algorithm based on deep learning," *[Journal Name or Conference if available]*, 2024, retrieved from uploaded document.

[21] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *European Conference on Computer Vision (ECCV)*. Springer, 2016, pp. 21–37. [Online]. Available: https://arxiv.org/abs/1512.02325

[22] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017. [Online]. Available: https://doi.org/10.1109/TPAMI.2016.2577031

[23] J. Quenum, K. Wang, and A. Zakhor, "Fast, accurate barcode detection in ultra high-resolution images," *arXiv preprint arXiv:2102.06868*, 2021. [Online]. Available: https://arxiv.org/abs/2102.06868

[24] K. He, G. Gkioxari, P. Dollár, and R. B. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2017, pp. 2961–2969. [Online]. Available: https://doi.org/10.1109/ICCV.2017.322

[25] M. S. Aslan, A. E. Cinar, M. Ozbayoglu, and O. G. Olmezogullari, "Performance analysis and cpu vs gpu comparison for deep learning," *ResearchGate*, 2019. [Online]. Available: https://www.researchgate.net/publication/334168063_Performance_Analysis_and_CPU_vs_GPU_Comparison_for_Deep_Learning

[26] M. D. Abo, "An efficiency comparison of npu, cpu, and gpu when executing an object detection model," 2023. [Online]. Available: https://kth.diva-portal.org/smash/get/diva2%3A1886212/FULLTEXT01.pdf

[27] NVIDIA, "Geforce rtx 30 series graphics cards," n.d., accessed: 2025-02-11. [Online]. Available: https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/

[28] ——, "Nvidia a100 tensor core gpu," n.d., accessed: 2025-02-11. [Online]. Available: https://www.nvidia.com/en-us/data-center/a100/

[29] ——, "Geforce rtx 40 series graphics cards," n.d., accessed: 2025-02-11. [Online]. Available: https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/

[30] G. Cloud, "Cloud tpu," n.d., accessed: 2025-02-11. [Online]. Available: https://cloud.google.com/tpu

[31] NVIDIA, "Jetson nano developer kit," n.d., accessed: 2025-02-11. [Online]. Available: https://developer.nvidia.com/embedded/jetson-nano-developer-kit

[32] ——, "Jetson xavier nx developer kit," n.d., accessed: 2025-02-11. [Online]. Available: https://developer.nvidia.com/embedded/jetson-xavier-nx-developer-kit

[33] ——, "Jetson agx orin developer kit," n.d., accessed: 2025-02-11. [Online]. Available: https://developer.nvidia.com/embedded/jetson-agx-orin-developer-kit

[34] AMD, "Rocm - open software platform for hpc and ai," n.d., accessed: 2025-02-11. [Online]. Available: https://rocmdocs.amd.com/en/latest/

[35] Intel, "Intel movidius neural compute stick," n.d., accessed: 2025-02-11. [Online]. Available: https://software.intel.com/content/www/us/en/develop/hardware/neural-compute-stick.html

[36] O. Michel, "Webots™: Professional mobile robot simulation," *International Journal of Advanced Robotic Systems*, 2004, cyberbotics Ltd.; developed in collaboration with EPFL.